



**university of
 groningen**

**faculty of science
and engineering**

Natural Language Processing Homework III

**Kim Wattez: S5499283
Emily Heugen: S5587042
Maurice Theo Meijer: S5480604
Group 15**

March 30, 2025

1 Sentiment analysis

Question a)

The generated plots that show the attention masking patterns.

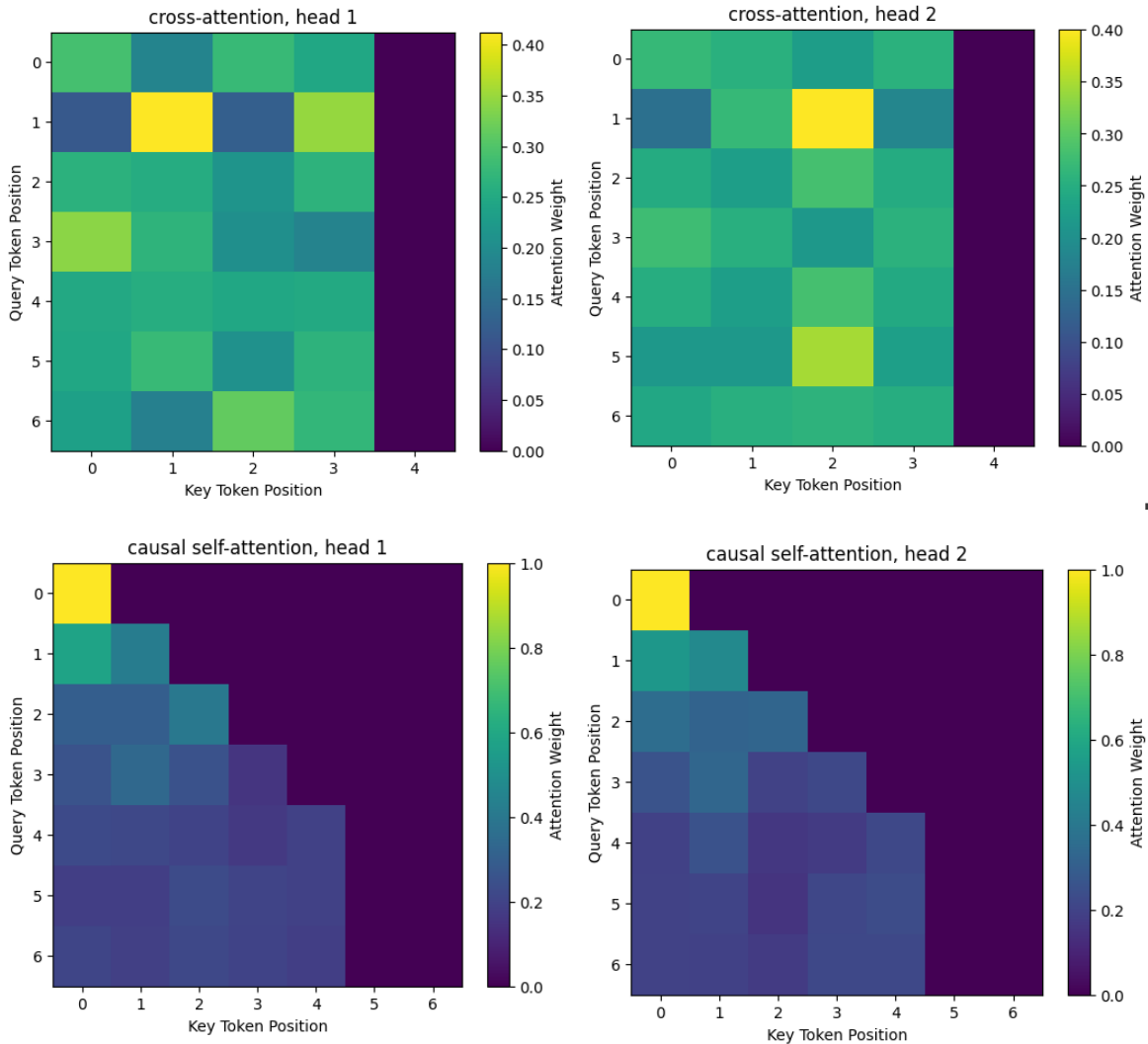


Figure 1: Attention masking patterns

Question b)

i)

The vocabulary size including special tokens is 3200 for both source language and target language. The amount of source language tokens can be read in the Model architecture under source_embeddings and then token_Embeddings in the training log. The target language amount will be found in the Model architecture under target_embeddings and then token_Embeddings in the training log. This part of the training log is shown in Figure 2

```

Model architecture: EncoderDecoderModel(
  (source_embeddings): TransformerEmbeddings(
    (token_embeddings): Embedding(3200, 32)
    (positional_embedding): Embedding(32, 32)
  )
  (target_embeddings): TransformerEmbeddings(
    (token_embeddings): Embedding(3200, 32)
    (positional_embedding): Embedding(32, 32)
  )
)

```

Figure 2: Training log showing the amount of source and target tokens

ii)

We modified the training function of the models in the code to print the amount of tokens, pad tokens and the proportions of pad tokens in each batch. This gives us an average amount of 380 tokens in each training batch. It also gives the proportion of pad tokens which is around 30% per batch. Figure 3 shows this part of the training log.

```

epoch 1 (109/272) | step 110 | avg_nll=28.37 avg_ppl=49.22
speed=1928.68 words/sec time_elapsed=7.66 sec tokens=352 pad_tokens=87
proportion_padding=24.71590909090909
epoch 1 (119/272) | step 120 | avg_nll=27.79 avg_ppl=43.58
speed=4023.72 words/sec time_elapsed=8.24 sec tokens=384 pad_tokens=121
proportion_padding=31.510416666666668
epoch 1 (129/272) | step 130 | avg_nll=25.54 avg_ppl=35.40
speed=4351.61 words/sec time_elapsed=8.77 sec tokens=384 pad_tokens=123
proportion_padding=32.03125
epoch 1 (139/272) | step 140 | avg_nll=26.70 avg_ppl=38.06
speed=4159.73 words/sec time_elapsed=9.33 sec tokens=384 pad_tokens=113
proportion_padding=29.427083333333332

```

Figure 3: Training log showing the amount of tokens per training batch

iii)

Saving the model parameters throughout training ensures that you can continue training a model if the training gets interrupted. If something happens during the model training, you can continue training from the last save instead of starting over.

It also allows us to load the parameters into a new model instance so we can evaluate the performance. This is what it is mostly used for in the given code.

Question c)

Corpus BLEU: 54.84682843569355

This score indicates that the machine translation system produces fluent, high-quality translations that closely resemble human-translated references. According to industry standards, a BLEU score

in the 50–60 range suggests that translations are generally accurate and of high quality. However, a manual review allows us to pinpoint both the strengths and weaknesses of the model's performance.

Reference: <https://cloud.google.com/translate/docs/advanced/automl-evaluate>

The model often translates sentence subjects correctly and frequently handles “to be” verbs and negation with accuracy. In some instances, its output perfectly matches the reference translations, as seen in Sample 11. However, there are recurring errors that reveal areas for improvement. For example, verbs are often mistranslated, such as in Sample 16, where:

je pars aujourd'hui (Gold: “I’m leaving today.”)

was incorrectly rendered as “I’m dying.”

Similarly, misinterpretations of meaning occur, as in Sample 14, where:

vous allez perdre (Gold: “You’re going to lose.”)

Was translated as “You’re kidding.”

Another common issue is incorrect pronoun or object selection, as seen in Sample 10, where:

vous me mettez mal à l'aise (Gold: “You’re embarrassing me.”)

Was translated as “You’re hurting him.”

While most translations form grammatical English sentences, some contain errors that affect fluency and meaning. An example is Sample 12, where the model produced the ungrammatical phrase:

“she’s roll well now.”

Listed below are samples 10 through 19 of the model's translations.

===== Sample 10 =====

Input: vous me mettez mal à l'aise .

Gold: you re embarrassing me .

Pred: you re hurting him .

===== Sample 11 =====

Input: c'est toi le professeur .

Gold: you re the teacher .

Pred: you re the teacher .

===== Sample 12 =====

Input: elle sourit avec bonheur .

Gold: she smiled happily .

Pred: she s roll well now .

===== Sample 13 =====

Input: je ne suis pas devin .

Gold: i m not a psychic .

Pred: i m not listening .

===== Sample 14 =====

Input: vous allez perdre .

Gold: you re going to lose .

Pred: you re kidding .

```
===== Sample 15 =====
Input:  vous me touchez .
Gold:   you re touching me .
Pred:   you re winning .
===== Sample 16 =====
Input:  je pars aujourd hui .
Gold:   i m leaving today .
Pred:   i m dying .
===== Sample 17 =====
Input:  je ne suis pas une sainte .
Gold:   i m no saint .
Pred:   i m not a doctor .
===== Sample 18 =====
Input:  je suis puissant .
Gold:   i m powerful .
Pred:   i m exhausted .
===== Sample 19 =====
Input:  vous n etes qu un lache .
Gold:   you re nothing but a coward .
Pred:   you re such a coward .
```

The model performs well overall, achieving a high BLEU score and generating mostly fluent translations. However, it struggles with verb meanings, word choices, and certain idiomatic expressions. Improving contextual understanding and handling of ambiguous words could further enhance translation quality.

Question d)

The beam search in the provided code is inefficient due to the repeated computation of encoder outputs at each decoding step. Since the encoder output remains unchanged throughout the sequence generation, recalculating it is unnecessary and costly. Additionally, the decoder does not utilize cached past key-value pairs, leading to redundant self-attention computations.

If we were to optimize this, the encoder should be called only once at the start, and the output it generates should be re-used throughout the decoding process. The decoder should be modified to return and utilize cached past key-value pairs to avoid recomputing attention over previously generated tokens. Moreover, the beam search could manage expanded sequences efficiently by maintaining only the most promising candidates while reducing unnecessary tensor operations. These improvements would enhance computational efficiency and decoding speed without sacrificing translation quality.

2 Comparison with LLM

2.1 Model Picking

We looked at both the MarianMT and T5 models to determine which one to use as a comparison model for our own model. MarianMT is designed specifically for neural machine translation and T5 is a more general-purpose language model which can perform tasks beyond pure translation, like summarization.

We chose to compare our model to MarianMT and not T5 because of the following reasons. First, MarianMT is pre-trained and fine-tuned only for translation, making it more efficient and accurate for this task than T5, which is more general and designed for a wider range of applications. Second, because MarianMT is focused only on translation, it is computationally more efficient and less complex. T5 has for this task a unnecessary complexity because it is also build to do other tasks, making the model itself more complex, which as a result leads to more computation before a good result is being reached.

Additionally, MarianMT is optimized using OPUS datasets, a large-scale multilingual corpus made for machine translation. This makes a reliable model for evaluating our model's translation capabilities. In contrast, T5 requires additional fine-tuning to achieve similar performance levels in translation.

This all shows that for the purpose of comparison a model to our own, MarianMT is the better option.

2.2 Experimental setup

To preprocess the input, we use `MarianTokenizer`, which tokenizes the source sentences while ensuring proper formatting. The input sentences are tokenized as follows:

```
tokenizer(texts , return_tensors="pt" , padding=True , truncation=True)
```

where:

- **Padding** ensures that all input sequences in a batch have the same length.
- **Truncation** prevents sequences from exceeding the model's maximum length.
- **Return tensors** converts tokenized input into PyTorch tensors.

For translation, the model generates output sequences the following code based on greedy search:

```
translated = model.generate(**inputs)
```

To evaluate the model quality, we compute the **BLEU score** using:

```
bleu_score = corpus_bleu([[ref] for ref in reference_sentences],
                           candidate_sentences)
```

The same test dataset is used to evaluate both **MarianMT** and our own model. After computing BLEU scores, we compare the translations of both models. Five example translations are displayed from both **MarianMT** and our own model.

2.3 Quantitive evaluation

MarianMT ended up with a BLEU-score of 56.91... after training and testing.

2.4 Qualitative Analysis

In our comparison we look at fluency, grammatical correctness and adequacy of the translation. We look at the translations of the sentences found in Figure 1

Input (French)	Reference (English)	Your Model	MarianMT
<i>on peut me faire confiance.</i>	I'm trustworthy.	We're being black-mailed.	I can be trusted.
<i>je ne suis pas malheureux.</i>	I'm not miserable.	I'm not friends.	I'm not unhappy.
<i>il adore les chats.</i>	He's a cat lover.	He's looking for the moment.	He loves cats.

Table 1: Comparison of Translations

For the first sentence, *On peut me faire confiance*, both translations are grammatically correct and fluent. Unfortunately, Our model misinterpreted the phrase, leading to an semantically incorrect translation, whereas MarianMT's output accurately translated the sentence with the correct meaning.

The second sentence, *Je ne suis pas malheureux*, is mostly translated correctly by both models. However, our model incorrectly translates *malheureux* as "friends" instead of "miserable". On technicallity the translation is still grammatical, but it is not fluent nor adequate. MarianMT on the other hand translates the sentence correctly, though it uses "unhappy" instead of "miserable." While this is a different word, the meaning remains largely intact since both words are quite similar.

In the final sentence, *Il adore les chats*, our model completely misinterprets the original meaning. The translation remains fluent and grammatical but is entirely inadequate as it has no connection to the original sentence. MarianMT produces a translation that is a bit different than the reference sentence but is still correct in terms of grammar, fluency, and adequacy.

2.5 Discussion

When comparing MarianMT to our own model, we see that MarianMT achieves a higher BLEU score of 56.91... compared to our model's 54.84.... The difference is relatively small at just 2.07.... This can also be seen when manually sorting through the translations. Both models produce mostly fluent and grammatically correct sentences. The difference is that MarianMT is more accurate in capturing the correct meaning, compared to our model, which struggles sometimes. We believe it is due to the fact that our model is very small.

In terms of computational requirements, training our own model was actually less expensive than using MarianMT. Our model completed training in just 4 minutes, even with multiple epochs and hyperparameter tuning, where processing the same data with MarianMT took 5 minutes despite

being pretrained.

The main trade-off between the two models lies in ease of use versus customization and understanding. MarianMT, requires minimal effort to use and provides mostly correct translations due to its extensive pretraining. However, it offers limited flexibility and understanding of how the model works. In contrast, our model, despite its slightly lower accuracy, allows for fine-tuning on smaller datasets, making it more adaptable for specialized tasks. It is also easier to modify our own model and this provides better insight into how the architecture of the model looks and how it works, making it easier to adjust if necessary.

Overall, MarianMT is the better choice for general-purpose translation due to its slightly higher accuracy and lower development effort, since you do not have to build the model from scratch. However, our model is more computationally efficient, easier to modify, and offers better understanding, making it a good alternative in scenarios where customization and adaptability are important.